

Hunter College
Department of Computer Science

CSCI 39548: Practical Web Development

Summer 2026

Assignment 4 — MyShelf (Router · Zustand · TanStack Query)

Instructor: *Sourya Saha* (sourya.saha24@login.cuny.edu)

Weight	20% of course grade
Released	Monday, June 29, 2026
Due	Monday, July 6, 2026, 11:59 PM
Submission	Brightspace — GitHub repo URL + live URL (optional)
AI	Allowed and encouraged — see policy below
Type	Individual

1 What You’re Building

MyShelf is a personal tracker for anything you consume — movies, books, shows, podcasts, courses, video games. You pick the theme; the app shape is the same.

The user can:

1. **Browse a catalog** of items seeded from a JSON file.
2. **Filter by status** — *want / active / done / dropped*.
3. **Search by title** with the query in the URL (shareable / refresh-survives).
4. **Open a detail page** for each item.
5. **Change an item’s status**, add a **note**, and give it a **1–5 rating**.
6. **Toggle a theme** (light/dark) and **density** (compact/comfortable) that persist across reloads.

You write the React app. The “backend” is `json-server` over a `db.json` file — same pattern from the Build Your Own Backend deck.

Pick a theme that excites you. “MyShelf” of books, “WatchPile” of movies, “CourseQueue” of MOOCs, “GameVault” — same app, your wrapper. The grader will read your README to know what you built.

2 The Three Pillars

Each pillar is a primary topic from the course. You must use **all three** correctly.

2.1 Pillar 1: React Router (20 pts)

2.1.1 Required Routes

Path	Purpose	Notes
/	Catalog (all items)	Default landing page
/items/:id	Item detail page	Reads id with <code>useParams</code>
/list/:status	Filtered by status	<code>status</code> is one of: <code>want</code> , <code>active</code> , <code>done</code> , <code>dropped</code>
*	404 NotFound page	Catch-all, must be LAST in route table

2.1.2 Required URL State

The search box on the catalog (and on `/list/:status`) writes to `?q=...` via `useSearchParams`. Refreshing the page must restore the search. Pasting a search URL must work.

2.1.3 Required Navigation

- A persistent top nav with `<NavLink>`s — at least Home, each status filter, and an About page.
- Active link is visually distinct (Tailwind class change driven by `isActive`).

2.2 Pillar 2: TanStack Query (25 pts)

2.2.1 Required Queries

- `useQuery({ queryKey: ["items"], queryFn: ... })` — load the full catalog.
- `useQuery({ queryKey: ["items", id], queryFn: ... })` — load a single item by id on the detail page.

2.2.2 Required Mutations

- `useMutation` for **status change**, **note update**, and **rating update**.
- Each `onSuccess` calls `invalidateQueries({ queryKey: ["items"] })` so the list refetches.

2.2.3 Required Loading + Error UX

- Every page that uses `useQuery` must render a **loading state** while `isLoading` is true.
- Every page must render an **error state** when `isError` is true (don't crash; show a message).
- The detail page must show a "Not found" message if `data` is null/undefined after loading (i.e., bad id).

2.3 Pillar 3: Zustand (20 pts)

2.3.1 Required Store

A `useUiStore` (name it what you want) containing:

- `theme: "light" | "dark"` with `toggleTheme()`.
- `density: "compact" | "comfortable"` with `setDensity(density)`.

2.3.2 Required Middleware

Wrap the store with `persist` so theme + density survive reload. Use a stable name (e.g., `"myshelf.ui"`).

2.3.3 Required Selector Hygiene

- Read `theme` and `density` as **primitives** (or with `useShallow` if you select an object slice).
- At least one component must use a selector — no `useUiStore()` calls that return the whole store.

Note: No prop drilling for theme/density. Components that need them call the store directly.

3 The Data Layer

You write `db.json` yourself. The shape below is **mandatory** so the grader can read every submission with one mental model.

3.1 Schema

Top-level resource: `items`. Each item:

Field	Type	Rules
<code>id</code>	number	Required, unique, starts at 1
<code>title</code>	string	Required, non-empty
<code>creator</code>	string	Required, non-empty (director / author / artist)
<code>year</code>	number	Required, 1900–2030
<code>genre</code>	string	Required, non-empty
<code>status</code>	string	One of: <code>"want"</code> , <code>"active"</code> , <code>"done"</code> , <code>"dropped"</code>
<code>rating</code>	number null	null if not rated, else integer 1–5
<code>note</code>	string null	null if no note, else string

3.2 Minimum Content

- At least **20 items**.
- At least **3 items per status** value (so each `/list/:status` page has something).
- At least **3 distinct genres**.
- At least **5 different creator values**.

3.3 Example Rows

```
{
  "items": [
    {
      "id": 1,
      "title": "Dune",
      "creator": "Denis Villeneuve",
      "year": 2021,
      "genre": "Science Fiction",
      "status": "done",
      "rating": 5,
      "note": "Worth the IMAX."
    },
    {
      "id": 2,
      "title": "The Three-Body Problem",
      "creator": "Liu Cixin",
      "year": 2008,
      "genre": "Science Fiction",
      "status": "active",
      "rating": null,
      "note": "Slow start, picking up."
    }
  ]
}
```

```
}
```

Keep `db.json` and `db.seed.json` per the Build Your Own Backend pattern. `npm run reset-db` should restore the seed.

4 Project Setup

1. Bootstrap:

```
npm create vite@latest myshelf -- --template react-ts
```

Install dependencies:

```
npm install react-router-dom @tanstack/react-query
@tanstack/react-query-devtools zustand
npm install -D json-server@0.17.4
tailwindcss @tailwindcss/vite
```

2. Scripts in package.json:

```
"dev":      "vite",
"server":   "json-server --watch db.json --port 3001",
"reset-db": "cp db.seed.json db.json",
"build":    "tsc -b && vite build"
```

3. Two terminals. One runs `npm run dev` (Vite on `:5173`), the other runs `npm run server` (json-server on `:3001`).

4. Provider order in `main.tsx`:

```
BrowserRouter > QueryClientProvider > App
```

Router **outside** TanStack so the cache survives all navigations.

Note: No Express, no Prisma, no Postgres for A4. Those are for the final project.

5 Stretch Features (up to +20 bonus, cannot exceed 100)

Pick any combination. Document what you did in the README so the grader sees it.

Feature	Bonus	What's required
Optimistic updates on status change	+5	<code>useMutation</code> with <code>onMutate</code> + rollback on error
<code>useShallow</code> on array/object selector	+3	Code comment explaining the trap you avoided
Dependent query: “More like this” on detail page	+5	Use <code>enabled: !!data</code> to wait, then fetch related by genre
Deploy to Vercel + free json-server host	+5	Live URL works in incognito; README lists both URLs
Per-user view (<code>?owner=alice</code> query param)	+2	URL-driven, no auth required

Stretch is graded only if the base 100 pts are essentially complete.

6 Submission

Submit on **Brightspace**:

1. **GitHub repo URL**. Public, or invite the instructor's GitHub user (provided on Brightspace).
2. **Live URL** (only if you did the deploy stretch).
3. **A short note** (3–5 sentences) describing the theme you chose and any stretch features done.

The repo must contain:

- A working `npm install && npm run server` (terminal A) + `npm run dev` (terminal B) flow.
- A `db.json` and a `db.seed.json` with ≥ 20 items meeting the schema.
- A `README.md` with: project description, theme chosen, setup steps, screenshots (1–3 is fine), stretch features done.
- The `node_modules/` and `.env*` files **excluded** via `.gitignore`.

Note: Do not commit `node_modules`. Use the Vite default `.gitignore` — it's correct.

7 Grading Rubric (100 pts + up to 20 bonus)

Category	Pts	What earns full marks
Routing + URL state	20	All required routes exist; <code>useParams</code> reads correctly; <code>useSearchParams</code> drives search; <code>NotFound</code> works; <code>NavLink</code> active state visible
TanStack Query	25	List + detail queries; 3 mutations; <code>invalidateQueries</code> wires writes to reads; loading + error states on every page
Zustand store + persist	20	UI store exists; persist middleware works; selectors used; theme + density drive the UI
Tailwind UI + responsive	15	Usable on mobile + desktop; density + theme visibly change layout/colors; no broken layouts
README + setup	10	Clear theme description; setup steps work on a fresh machine; screenshots present
Code clarity	10	TypeScript types real (no <code>any</code>); files organized; imports clean
Stretch	+20	Per feature table above; capped at 100 total

7.1 Deductions

- **Sparse README** — minimum -4 . Escalates to -8 if essentially empty.
- **Minimal styling** — minimum -2 . Escalates to -5 if visually unfinished.
- **Hardcoded URLs scattered across files** — -2 . Use a single `const API_URL`.
- **useQuery results not type-narrowed** (`data.length` without guard) $\rightarrow$ runtime crash $\rightarrow -3$ per page.
- **No mutation invalidation** (list doesn't refresh) $\rightarrow -5$.
- **Theme/density not persisting** $\rightarrow -5$.
- **db.json doesn't meet schema rules** $\rightarrow -1$ per rule violated, capped at -10 .

8 AI Policy

AI tools (ChatGPT, Claude, Copilot, etc.) are **allowed and encouraged**. The expectation: **informed use**.

- You must be able to explain every line you submit, in your own words.
- If the instructor asks “why did you pick that selector shape?” or “why does this `queryKey` include the id?” — you should have an answer.
- Copy-paste without understanding will hurt you on the proctored exit assessment, where AI is **banned**.
- If you used AI heavily for some part, briefly say so in the README and what you learned. Not a deduction; just transparency.

9 Common Pitfalls

9.1 Router

- **NotFound route must be LAST.** `path="*" matches everything; if it's first, every route hits it.`
- `useParams` always returns strings. Convert with `Number(...)`.
- `<NavLink to="/">` highlights everywhere. Add `end` to the Home `NavLink`.
- **Routes inside `QueryClientProvider`, not outside.** Wrong order = cache cleared on every navigation.

9.2 TanStack Query

- `queryKey` per resource. `["items"]` for the list, `["items", id]` for the detail.
- `invalidateQueries` invalidates all matching queries. `queryKey: ["items"]` invalidates both the list and all detail queries starting with `["items", ...]`.
- data can be undefined. Guard: `data?.title` or `(data ?? [])`.
- `mutate` is fire-and-forget; `mutateAsync` returns a promise. Don't await `mutate()`.

9.3 Zustand

- **The useShallow trap.** A selector returning a fresh array/object every call → infinite loop.
- **partialize to skip actions.** Without it, `persist` tries to serialize functions.
- **Don't shove server data into Zustand.** The catalog lives in TanStack Query.

9.4 json-server

- **db.json rewrites itself on every mutation.** Reset with `npm run reset-db` before screenshots.
- **Port 3001, not 3000.** Vite owns 3000 in some configs.
- **json-server@0.17.4 — pinned.** v1.x has a different CLI.

9.5 General

- Don't ship `console.logs` or commented-out code blocks.
- TypeScript errors should be 0 on a fresh `npm run build`.

10 Getting Help

1. **Read the docs.** `react.dev`, `reactrouter.com`, `tanstack.com/query`, `zustand-demo.pmnd.rs`.
2. **Search the internet.** Every error message has been hit and answered.
3. **Use the Brightspace forum.** Posting helps classmates; answering makes you better.
4. **Discuss with peers.** Your final-project teammates are your first debugging line.
5. **Bring questions to class.** Office hours by appointment via email.

11 What This Assignment Rehearses

A4 is the smallest version of the final project's frontend. The final project adds a real Express + Prisma backend, a real deployment, group collaboration (2–3 person teams), and a larger feature surface.

If A4 goes smoothly, the final-project frontend is “more screens, same patterns.” If A4 is rough, the final project will be rougher — fix what you struggled with here before final-project week.

Good luck. Build something you'd want to show in an interview.

End of Assignment 4